

https://github.com/shubhkpatel/Low-Level-Design/tree/main/src/main/java/org/nailyourinterview/LLD/rate_limiter

convert the code of rate limiter to nodejs

Yes. I checked the repository you linked. The rate limiter is organized around **Strategy + Factory + Limiter + Model/Service** packages, and the core implementation uses a **Token Bucket** approach with per-client buckets.  [GitHub](#)

Below is a clean **Node.js + TypeScript** conversion that preserves the same LLD structure rather than simply translating Java syntax.

1. Java → Node.js mapping

Java	Node.js / TypeScript
<code>interface RateLimiter</code>	<code>IRateLimiter</code>
<code>enum</code>	TypeScript <code>enum</code>
<code>HashMap</code>	<code>Map</code>
<code>ConcurrentHashMap</code>	<code>Map</code> / Redis for distributed version
<code>Factory</code>	<code>RateLimiterFactory</code>
<code>Service</code>	<code>RateLimiterService</code>
<code>TokenBucket</code>	<code>TokenBucketLimiter</code>
<code>Main.java</code>	<code>main.ts</code>
POJO model	TypeScript <code>interface</code> / <code>class</code>

The important design idea is that the **RateLimiter does not know how the algorithm works**. It delegates the decision to a strategy. This makes Token Bucket replaceable with Sliding Window, Leaky Bucket, etc.  [AlgoMonster](#) +1

Recommended Node.js structure

```
rate-limiter/
├── src/
│   ├── enums/
│   │   └── rate-limiter-type.enum.ts
│   ├── models/
│   │   └── rate-limit-config.model.ts
│   ├── limiter/
│   │   ├── rate-limit-strategy.ts
│   │   ├── token-bucket.limiter.ts
│   │   └── sliding-window.limiter.ts
│   ├── factory/
│   │   └── rate-limiter.factory.ts
│   ├── service/
│   │   └── rate-limiter.service.ts
│   └── main.ts
├── package.json
└── tsconfig.json
```

rate-limiter-type.enum.ts

```
export enum RateLimiterType {
  TOKEN_BUCKET = 'TOKEN_BUCKET',
  SLIDING_WINDOW = 'SLIDING_WINDOW'
},
// TypeScript
```

rate-limit-config.model.ts

```
import { RateLimiterType } from '../enums/rate-limiter-type.enum';

export interface RateLimitConfig {
  type: RateLimiterType;
  capacity: number;
  windowMs: number;
```

2. Strategy

```
export interface RateLimitStrategy {  
  allow(clientId: string): boolean;  
}
```

</> TypeScript



This is the most important part of the LLD.

The service only knows:

```
RateLimitStrategy  
├── TokenBucketLimiter  
├── SlidingWindowLimiter  
└── Future algorithms
```



So adding another algorithm doesn't require modifying the service.

3. Token Bucket implementation

```
interface Bucket {  
  tokens: number;  
  lastRefill: number;  
}  
  
export class TokenBucketLimiter implements RateLimitStrategy {  
  
  private readonly buckets = new Map<string, Bucket>();  
  
  constructor(  
    private readonly capacity: number,  
    private readonly windowMs: number  
  ) {}  
}
```

```
allow(clientId: string): boolean {

    const now = Date.now();

    let bucket = this.buckets.get(clientId);

    // First request from this client
    if (!bucket) {
        bucket = {
            tokens: this.capacity,
            lastRefill: now
        };

        this.buckets.set(clientId, bucket);
    }

    this.refill(bucket, now);

    if (bucket.tokens > 0) {
        bucket.tokens--;
        return true;
    }

    return false;
}

private refill(
    bucket: Bucket,
    now: number
): void {

    const elapsed = now - bucket.lastRefill;

    if (elapsed <= 0) {
        return;
    }

    const refillRate =
        this.capacity / this.windowMs;

    const tokensToAdd =
        Math.floor(elapsed * refillRate);

    if (tokensToAdd <= 0) {
        return;
    }

    bucket.tokens = Math.min(
        this.capacity,
```

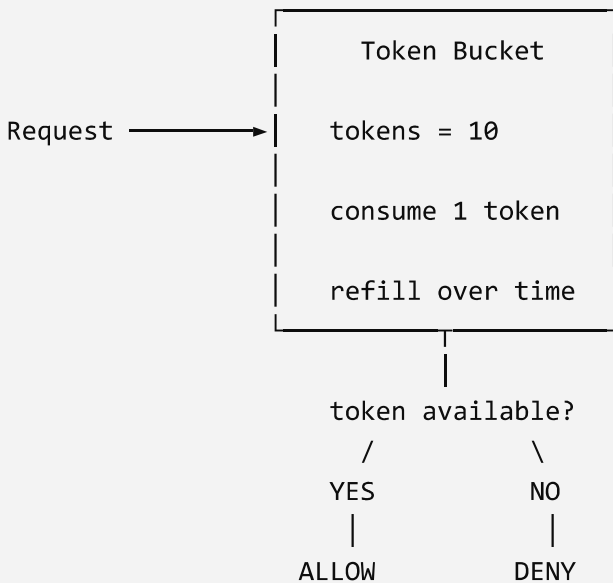
```

        bucket.tokens + tokensToAdd
    );

    bucket.lastRefill = now;
}
}

```

Conceptually:



4. Factory

The factory hides object creation from the service.

```

import { RateLimiterType } from '../enums/rate-limiter-type.enum';
import { RateLimitConfig } from '../models/rate-limit-config.model';
import { RateLimitStrategy } from '../limiter/rate-limit-strategy';
import { TokenBucketLimiter } from '../limiter/token-bucket.limiter';

export class RateLimiterFactory {

    static create(
        config: RateLimitConfig
    ): RateLimitStrategy {

        switch (config.type) {

            case RateLimiterType.TOKEN_BUCKET:

```

```
        return new TokenBucketLimiter(  
            config.capacity,  
            config.windowMs  
        );  
  
        default:  
            throw new Error(  
                `Unsupported rate limiter: ${config.type}`  
            );  
    }  
}
```

5. RateLimiterService

This is where the per-client isolation happens.

```
import { RateLimitConfig } from '../models/rate-limit-config.model';  
import { RateLimiterFactory } from '../factory/rate-limiter.factory';  
import { RateLimitStrategy } from '../limiter/rate-limit-strategy';  
  
export class RateLimiterService {  
  
    private readonly limiters =  
        new Map<string, RateLimitStrategy>();  
  
    constructor(  
        private readonly config: RateLimitConfig  
    ) {}  
  
    request(clientId: string): boolean {  
  
        let limiter = this.limiters.get(clientId);  
  
        if (!limiter) {  
  
            limiter =  
                RateLimiterFactory.create(this.config);  
  
            this.limiters.set(  
                clientId,  
                limiter  
            );  
        }  
    }  
}
```

```

    return limiter.allow(clientId);
  }
}

```

Notice the important design:

```

RateLimiterService
├── Map<ClientId, Strategy>
├── user-100 → TokenBucket
├── user-200 → TokenBucket
├── user-300 → TokenBucket
└── user-400 → TokenBucket

```

Therefore:

```

user-100
  ↓
Bucket A
  ↓
5 requests

user-200
  ↓
Bucket B
  ↓
5 requests

```

One client's requests don't consume another client's quota.

6. main.ts

```

<> TypeScript

import { RateLimiterService } from './service/rate-limiter.service';
import { RateLimiterType } from './enums/rate-limiter-type.enum';
import { RateLimitConfig } from './models/rate-limit-config.model';

const config: RateLimitConfig = {
  type: RateLimiterType.TOKEN_BUCKET,
  capacity: 5,
  windowMs: 10_000
};

```

```
const rateLimiter =
  new RateLimiterService(config);

const client = 'user-123';

for (let i = 1; i <= 10; i++) {

  const allowed =
    rateLimiter.request(client);

  console.log(
    `Request ${i}:`,
    allowed ? 'ALLOW' : 'DENY'
  );
}
```

You would get approximately:

Request 1: ALLOW
Request 2: ALLOW
Request 3: ALLOW
Request 4: ALLOW
Request 5: ALLOW
Request 6: DENY
Request 7: DENY
Request 8: DENY
Request 9: DENY
Request 10: DENY

✦ Upgrade

After enough time passes, tokens are refilled.

7. Express integration

For a real Node.js application, I would put the limiter in middleware.

</> TypeScript

```
import { Request, Response, NextFunction } from 'express';
import { RateLimiterService } from './service/rate-limiter.service';

export function rateLimitMiddleware(
  rateLimiter: RateLimiterService
) {
```



```
return (
  req: Request,
  res: Response,
  next: NextFunction
) => {

  const clientId =
    req.user?.id ??
    req.ip;

  const allowed =
    rateLimiter.request(clientId);

  if (!allowed) {

    res.setHeader(
      'Retry-After',
      '1'
    );

    return res
      .status(429)
      .json({
        error: 'Too Many Requests'
      });
  }

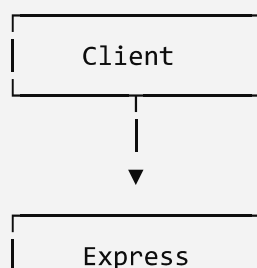
  next();
};
```

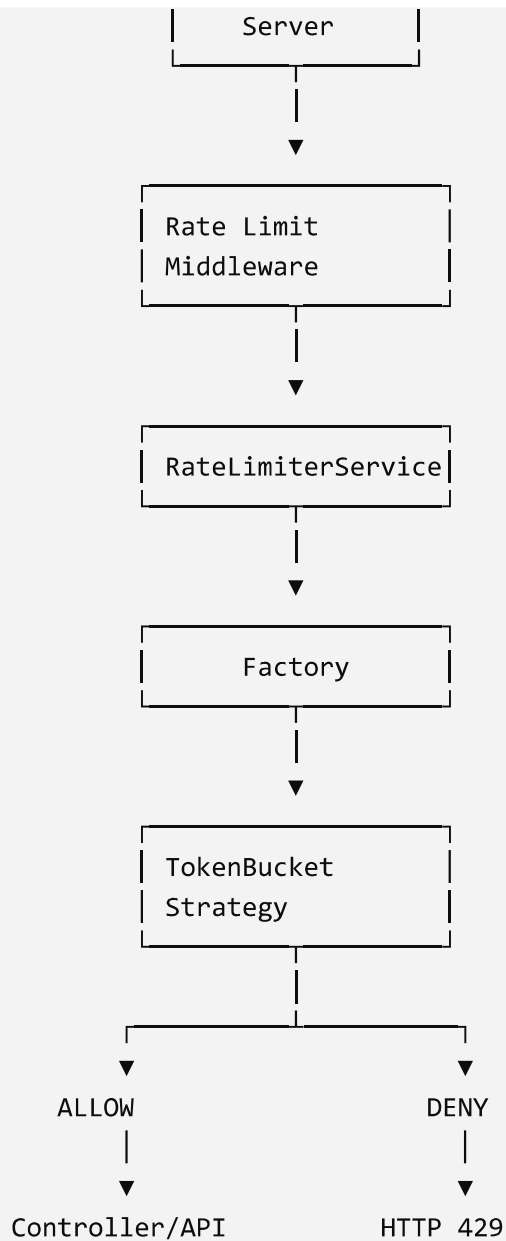
Then:

</> TypeScript

```
app.use(
  rateLimitMiddleware(rateLimiter)
);
```

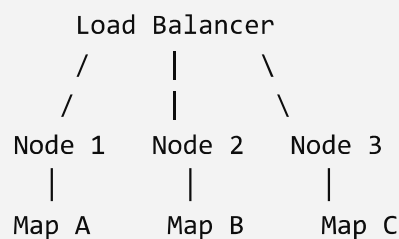
Now the architecture becomes:





One important production change

The above implementation is an **in-memory, single-process** conversion of the LLD. That is appropriate for understanding the original design, but it is **not sufficient when Node.js is running multiple instances**:



A user could effectively get:

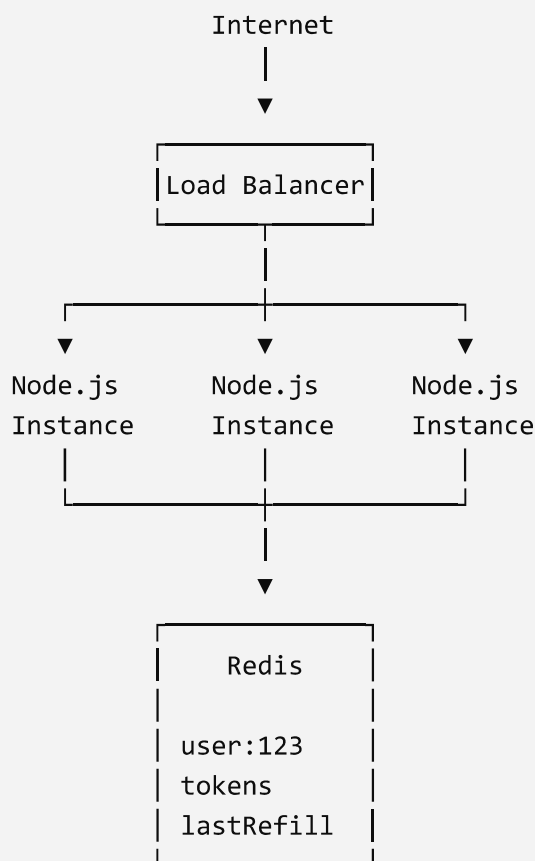
Node 1 → 100 requests
Node 2 → 100 requests
Node 3 → 100 requests



instead of the intended 100 total.

For a distributed production rate limiter, the usual next step is to move the bucket/window state into **Redis**, making the state shared across application instances. Distributed rate limiting is specifically different from the single-process LLD because the shared state and atomicity become central concerns. Low Level Design +1

So I would evolve the design to:



That version would also need **atomic Redis operations/Lua scripting**, because a distributed limiter cannot safely perform `GET → calculate → SET` as separate operations under concurrent requests.

The repository's original design is therefore a good **LLD foundation**, while the Redis version is what I'd use to demonstrate **production-level Node.js system design**. GitHub

If your goal is interview preparation, the most useful next step is to convert the **entire Java package one-to-one—including every** `enum` , `factory` , `limiter` , `model` , `service` , and `Main.java` **—into a runnable Node.js/TypeScript project**, and then add the **Redis distributed version** as Phase 2.



ChatGPT can make mistakes. Check important info.